

Agile Decision Architecture

Leveraging MBSE, Response Surface Methods, and Pareto Optimality

to Deliver Faster, More Defensible Engineering Decisions

A technical framework for compressing decision latency without sacrificing rigor

Preface: The Real Constraint Is Not Compute — It Is Decision Latency

Every engineering program has a binding constraint. In modern aerospace, defense, pharmaceutical, and industrial programs, that constraint is rarely computing power, tooling capability, or engineering talent. It is the time between when a consequential question arises and when a confident, defensible answer is available to the people who need to act on it.

Consider what happens in a typical program when the chief architect asks: "If we move the power module 15 centimeters forward, what happens to thermal margins, structural weight, and control system stability?" The answer that should arrive in hours instead arrives in weeks — because the thermal team, structures team, and controls team each need to rerun their analyses, reconcile their models, and resolve conflicting assumptions before any integrated answer is possible.

This latency is not a symptom of insufficient tooling. It is a symptom of disconnected tooling — three capable simulations that do not share a common model of the system, do not communicate their results in a form the others can consume, and do not connect their outputs to the formal requirements against which the decision must ultimately be judged.

The engineering programs that consistently deliver on schedule, within budget, and with fewer late-stage surprises are not the ones with the highest-fidelity individual simulations. They are the ones with the shortest cycle from engineering question to confident, integrated, requirements-traceable answer.

This document is a technical framework for building that capability. It shows how three well-established engineering methods — Model-Based Systems Engineering (MBSE) simulations, Response Surface Methods (RSM), and Pareto Optimality Analysis — function as a unified decision acceleration architecture when used together, and how to implement that architecture across the full program lifecycle.

The argument is structured around decisions, not tools. Each section opens with the decision problem it addresses, then develops the technical approach required to solve it. The technical depth is full — this is a practitioner document, not a management overview — but the organizing principle throughout is what capability each method enables at the moment a program needs to act.

PART I

The Decision Acceleration Architecture

Why three methods, used together, create a fundamentally different decision tempo

1. The Decision Triad: One Architecture, Not Three Methods

Decision outcomes in this section enable:

- Understand why each method is necessary but insufficient alone
- See the pipeline through which MBSE, RSM, and Pareto connect
- Identify which method addresses which class of engineering decision

1.1 The Question Each Method Answers

Each of the three methods answers a fundamentally different question. Understanding these distinctions prevents the most common misapplication — using one method where another is needed, or treating them as alternatives when they are complements.

Method	The Question It Answers	What It Produces	What It Cannot Do Alone
MBSE Simulation	What will this system actually do when all physics and behaviors interact?	Verified, requirements-traceable system behavior	Cannot efficiently sweep design space; too expensive per evaluation
Response Surface Methods	How does performance vary across my design space, and where is the optimum?	A cheap surrogate that can be swept in seconds	Cannot verify requirements; cannot handle multi-objective trade-offs
Pareto Optimality	What trade-offs are genuinely unavoidable, and what is the efficient frontier?	A structured decision surface for stakeholder choice	Cannot evaluate system behavior; needs data from the other two

The critical insight is that these three methods form a pipeline, not a menu. MBSE simulation provides the high-fidelity physics ground truth. RSM samples are strategically simulated to build a surrogate that can be evaluated thousands of times at negligible cost. The surrogate is then swept by a Pareto optimizer to map the trade-off frontier. Stakeholders select from the frontier with full awareness that it is grounded in validated physics — not in opinion, not in a single design point, and not in somebody's spreadsheet.

1.2 The Decision Latency Problem in Concrete Terms

Without this pipeline, the typical program decision cycle looks like this: a design change is proposed, each discipline reruns its analysis independently, results are reconciled in a review meeting, conflicts are resolved through negotiation rather than computation, and a decision is made that may or may not reflect the true physics of the integrated system. This cycle takes weeks and produces decisions of uncertain quality.

With the pipeline, the cycle looks like this: the surrogate model is queried instantly, the Pareto frontier is available in the room, and the decision conversation focuses on which point on the frontier best serves program objectives — a conversation that can happen in hours, grounded in physics that was validated once rather than repeatedly renegotiated.

10-50x Faster design space exploration <i>Surrogate vs. direct simulation sweeps</i>	60-80% Reduction in physical prototype iterations <i>When simulation fidelity meets verification standards</i>	3-5x More design alternatives evaluated <i>Per program phase with RSM+Pareto</i>
---	---	---

1.3 The Architecture in One Diagram

The pipeline flows in one direction for computation and in the reverse direction for requirements verification. This bidirectionality is what makes it a decision architecture rather than just a simulation workflow:

Flow direction	Stage 1	Stage 2	Stage 3	Stage 4
Computation (forward)	SysML v2 defines architecture and interfaces	MBSE simulation (Modelica, FEA, AADL) computes physics	RSM builds a surrogate from the DoE-sampled simulation	Pareto sweeps the surrogate to produce the frontier
Verification (reverse)	Pareto selection traces to stakeholder objectives	Selected design verified against RSM surrogate	Surrogate spot-checked against full MBSE simulation	Results traced to SysML v2 requirements via REST API

The verification reverse-flow is what elevates this from a simulation workflow to a decision architecture. Every point on the Pareto frontier can be traced back to the formal requirements it satisfies. When a stakeholder selects a design, they are not choosing a simulation result — they are choosing a requirements-traceable system configuration.

2. Response Surface Methods: Compressing Exploration Cost

Decision outcomes in this section enable:

- Select the appropriate DoE design for a given decision context
- Understand when the surrogate is trustworthy and when it is not
- Connect RSM output directly to Pareto optimization input

2.1 The Decision Problem RSM Solves

The decision problem is straightforward: the system model is too expensive to evaluate at every point in the design space that would be needed to find the optimum by brute force, but design decisions must be made before you can afford to wait for exhaustive analysis. RSM resolves this tension by running a small, strategically chosen set of evaluations, fitting a mathematical surface to the results, and optimizing the surface instead of the system.

The keyword is "strategically." Random sampling is not RSM. RSM uses the Design of Experiments theory to choose evaluation points that maximize information about the response surface per unit of experimental or computational cost. A well-designed experiment extracts essentially the same information as a full factorial sweep at a fraction of the evaluation count.

2.2 The Mathematical Surface

RSM fits a second-order polynomial to the sampled data. For two design variables x_1 and x_2 :

$$\hat{y} = \beta_0 + \beta_1x_1 + \beta_2x_2 + \beta_{12}x_1x_2 + \beta_{11}x_1^2 + \beta_{22}x_2^2$$

The six terms capture: a constant baseline, two linear main effects, one two-way interaction (does the effect of x_1 depend on the value of x_2 ?), and two pure quadratic curvature terms (is there a peak or valley in the design space?). These are estimated by ordinary least squares from the sampled data. The design matrix — the structured set of evaluation points — determines estimation quality.

For k design variables the model has $(k+1)(k+2)/2$ terms: 6 for two variables, 10 for three, 15 for four. This growth sets the practical ceiling. Beyond 6-7 variables, run counts to estimate all terms become prohibitive, and Kriging (Gaussian Process regression) or other machine-learning surrogates are better choices — they scale better dimensionally but require more validation effort.

2.3 Design Selection: Matching the Design to the Decision

The choice of DoE design is a decision in itself, driven by what you need to know and what you can afford to run:

Design	Run Count (k=3)	Strengths	When to Choose It
--------	--------------------	-----------	-------------------

Central Composite (CCD)	20	Estimates all quadratic terms; excellent prediction at design space boundaries	Default choice when boundary behavior matters; when all factor combinations are feasible
Box-Behnken	13	Avoids extreme corner combinations; efficient for 3-4 factors	When extreme factor combinations are infeasible, dangerous, or cost-prohibitive
Full factorial (reference)	125	Complete information; no assumptions	Computationally cheap responses; used as validation benchmark, not for surrogate building
Kriging + LHS	Flexible	Handles nonlinearity and discontinuities; scales to higher dimensions	When response is suspected to be highly nonlinear or when $k > 6$

2.4 Validation: When to Trust the Surrogate

A surrogate that has not been validated is a liability, not an asset. Validation requires three checks that every RSM practitioner must perform before using the surface for decision-making:

- **Lack-of-fit test:** Statistically tests whether the quadratic model adequately represents the data, or whether significantly higher-order terms are present. A significant lack-of-fit is a signal to add center points, transform the response, or move to a higher-order surrogate.
- **Cross-validation (leave-one-out):** Remove each design point in turn, refit the surface, and predict the removed point. The prediction error distribution quantifies how well the surrogate generalizes to unseen design space regions.
- **Confirmation runs:** Run 3-5 new evaluations at the predicted optimum and at the Pareto frontier candidates before committing to a design decision. If the surrogate predicts well at these points, it can be trusted. If it does not, the design space is more complex than the polynomial assumed.

Critical boundary condition

A surrogate is only valid within the design space where it was fitted. Using it to predict behavior at factor values outside the original experimental region — extrapolation — produces results that may appear numerically reasonable while being physically meaningless. Every decision made from a surrogate must be explicitly checked against this boundary.

2.5 Connecting RSM to Pareto: The Surrogate Sweep

Once a validated surrogate exists for each system objective, the Pareto optimizer sweeps all surrogates simultaneously. For a three-objective problem — minimize mass, minimize cost, maximize reliability — the optimizer evaluates thousands of design configurations in seconds using the surrogates, identifies the non-dominated set, and presents a three-dimensional Pareto surface to the decision team.

The decision team then selects candidate designs from the frontier. Those candidates — typically 3-5 points representing different trade-off positions — are verified against the full MBSE simulation. If the verification results match the surrogate predictions within acceptable tolerances, the decision is made. If not, the surrogate is refined in the regions where the prediction error is unacceptable.

This verify-then-decide workflow is what separates responsible use of surrogates from naive use. The surrogate accelerates exploration; the full simulation verifies the decision. Neither alone provides both speed and confidence.

3. Pareto Optimality: Replacing Opinion with Structured Trade-Off

Decision outcomes in this section enable:

- Replace unstructured design review debates with Pareto-grounded trade-off conversations
- Select from the frontier using methods appropriate to the decision context
- Identify knee points and non-convex regions that reveal design paradigm boundaries

3.1 The Decision Problem Pareto Solves

Most engineering decisions involve multiple objectives that are genuinely in conflict. Lighter structures are less stiff. Faster schedules increase cost. Higher reliability requires more redundancy, which adds weight. In unstructured design reviews, these conflicts are resolved through negotiation, seniority, or organizational inertia — not through explicit reasoning about what is physically possible and what must be traded away.

Pareto analysis replaces that negotiation with a structured decision surface: the Pareto frontier. Every point on the frontier represents a design that cannot be improved on any objective without degrading another. Every point below the frontier represents a design that is dominated — there exists at least one other design that is better on all objectives simultaneously. Choosing a dominated design is never rational. Choosing between non-dominated designs requires explicit value judgments — and Pareto analysis makes those value judgments visible and auditable.

The Pareto frontier does not make the decision. It eliminates the decisions that do not need to be made — all dominated designs — and structures the one conversation that does need to happen: which combination of trade-offs best serves program objectives? This is a fundamentally different conversation from "which design does the lead engineer prefer?"

3.2 The Geometry of Dominance

Solution A dominates solution B if A is at least as good as B on every objective and strictly better on at least one. This definition does not require weights or preferences — dominance is an objective relationship determined entirely by the objective function values. The frontier is the complete set of non-dominated solutions.

For discrete candidate solution sets, frontier construction tests every pair for dominance and retains only the non-dominated points. For continuous design spaces, Evolutionary Multi-objective Optimization (EMO) algorithms — principally NSGA-II for two and three objectives and NSGA-III for four or more — evolve a population toward the frontier through selection, crossover, and mutation, using Pareto rank and crowding distance rather than a scalar fitness function.

3.3 Reading the Frontier

The frontier's shape carries engineering information that practitioners should actively interpret:

Frontier Shape	What It Signals	Decision Implication
Smooth convex curve	Trade-offs are gradual and consistent throughout the space	Weighted sum optimization reliably finds the correct solution; weights can be specified a priori
Strongly curved (knee)	One region offers high simultaneous value on all objectives	The knee point is the rational default when stakeholder preferences are unclear
Non-convex region	Some trade-off combinations require designs that weighted sums cannot reach	Reference point methods or epsilon-constraint approaches are required for complete coverage
Discontinuous (gap)	Two distinct design paradigms exist with no smooth transition between them	Programs choosing hybrid designs may be selecting provably inferior solutions
Flat along one axis	One objective is insensitive to design choices in this region	More design freedom exists than expected; relaxing that constraint enables optimization elsewhere

3.4 Selecting from the Frontier: Methods and Trade-offs

Weighted sum scalarization

Assign weights reflecting stakeholder priorities, compute a weighted score for each frontier point, and select the maximum. This requires weights to be specified before seeing the frontier, which assumes stakeholders know their preferences before understanding the trade-off structure. It works well when preferences are stable and well-articulated. It systematically misses non-convex frontier regions. Use it when the frontier is expected to be convex and stakeholder preferences are clear.

Reference point methods (NSGA-III, epsilon-constraint)

The decision team specifies aspiration levels — "we need at least 85% reliability and at most \$2M unit cost" — and the optimizer finds frontier points closest to that reference. This is more natural for engineers who think in requirements rather than abstract weights. It is the standard approach for programs operating under formal requirement thresholds.

Knee point selection

Identify the frontier point with maximum curvature — the point where giving up a small amount on one objective yields the largest gain on another. This is the rational choice when stakeholders cannot articulate preferences before seeing the frontier. For most engineering programs encountering Pareto analysis for the first time, knee point selection is the practical starting point.

Hypervolume as a quality metric

When comparing two optimization runs, or tracking convergence, the hypervolume indicator quantifies frontier quality as a single scalar: the volume of objective space dominated by the frontier relative to a reference worst point. Larger hypervolume means either better individual solutions, more spread along the frontier, or both. This is how algorithm developers benchmark optimization methods and how program teams verify that their multi-objective optimization has converged.

PART II

The MBSE Simulation Engine

The technical infrastructure that grounds surrogate models and decision evidence in validated physics

4. MBSE Architecture: The Authoritative System Model

Decision outcomes in this section enable:

- Use SysML v2 to make control requirements formally verifiable rather than documentarily documented
- Connect the architecture model to simulation through typed interfaces and the REST API
- Understand when to route to SysML vs. AADL for different architectural concerns

4.1 Why the Architecture Model Matters for Decision-Making

The architecture model is not an interesting artifact for its own sake. It is interesting because it is the system's authoritative semantic contract — the document that says what the system is, what interfaces it presents, what requirements it must satisfy, and how behavior is allocated across physical components. When that contract is formal enough to be queried programmatically and precise enough to carry machine-verifiable constraints, every downstream simulation, every verification test, and every Pareto frontier candidate can be checked against it automatically.

Without a formal architecture model, the connections between design decisions and requirements are maintained in documents and spreadsheets. Changes propagate through email. Conflicts surface in program reviews. The cost of each decision includes the overhead of manually verifying that nothing has been violated upstream.

With a formal architecture model accessible through a standard API, those connections are structural. A change committed to the architecture model propagates immediately to every tool that queries it. Conflicts surface at commit time. The cost of each decision is a query, not a review cycle.

4.2 SysML v1 vs. v2: The Decision Velocity Difference

SysML v1, designed in 2006 as a UML profile, established systems modeling as a formal discipline. It introduced nine diagram types covering requirements, structure, and behavior, and built a mature ecosystem of tools — Cameo Systems Modeler, IBM Rhapsody, Enterprise Architect — with large installed bases.

The limitation that matters most for decision velocity is tooling lock-in. SysML v1 models live in proprietary XMI files that only the originating tool can faithfully read. Model changes do not propagate automatically. Round-tripping between tools loses fidelity. Querying the model programmatically — to check a simulation configuration against a formal interface declaration, for example — requires tool-vendor APIs that differ between products.

SysML v2, standardized in 2024, resolves this through three architectural commitments that directly affect decision tempo:

SysML v2 Feature	Technical Mechanism	Decision Velocity Impact
Normative textual syntax	Models are written as text, stored in Git, diffed line-by-line	Architecture changes are trackable, reviewable, and auditable at commit granularity
REST API repository	Standard HTTP endpoints for element CRUD and query	Any tool can query the architecture model without vendor adapters
Formal constraint requirements	RequirementUsage carries machine-evaluable constraint expressions	Simulation results can be checked against requirements automatically — no human comparison needed
Strong typing with ISO 80000 units	Port declarations carry quantity kinds and physical units	Interface mismatches (mass vs. force, Celsius vs. Fahrenheit) are caught at model assembly, not at test
Occurrence semantics	Elements have temporal life histories natively	Structural model connects directly to time-domain simulation without translation

4.3 When to Use AADL Instead of SysML

SysML v2 is the right tool for system-level architecture — requirements, function allocation, interface definitions, behavioral specification. AADL (Architecture Analysis and Design Language, SAE AS5506) is the right tool when the primary architectural question is: "Will this software execute correctly on this hardware within these timing constraints?"

AADL generates mathematical proof of schedulability — a certificate that all threads will meet their deadlines under the declared scheduling policy. This capability has no equivalent in SysML. For avionics programs operating under DO-178C, automotive programs under ISO 26262, and any safety-critical embedded system requiring formal timing verification, AADL is not optional — it is the only tool that produces the required class of evidence.

The mature integration pattern uses both: SysML v2 at the system level for function allocation and interface definition, AADL at the software architecture level for scheduling and safety proof, with traceability relationships connecting them so that a SysML v2 function allocation query can navigate to the AADL thread that implements it.

4.4 The REST API as Decision Infrastructure

The SysML v2 REST API is the technical mechanism that makes the architecture model a live participant in the decision pipeline rather than a static artifact consulted occasionally. Key endpoint patterns:

```
GET /projects/{id}/commits/{commitId}/elements
  - retrieve all model elements at a specific architecture version

POST /projects/{id}/commits
  - commit architecture changes; every change is versioned and auditable
```

```
POST /projects/{id}/commits/{commitId}/query-results
  - filter elements by type, name, or property for automated checks
```

The commit-based version model mirrors Git semantics. A simulation configuration validated against architecture commit abc123 is explicitly tied to that version of the architecture. When the architecture changes, the system knows which simulations and verification results are now stale — not because someone remembered to check, but because the version reference is structural.

5. Physics Simulation: Modelica and Multi-Domain Co-Simulation

Decision outcomes in this section enable:

- Choose the right physics simulation approach for the dominant coupling in your system
- Understand where FMI co-simulation is necessary and where it breaks
- Connect simulation outputs to RSM inputs through structured sampling

5.1 Why Multi-Domain Physics Is the Core Capability

The decision problems that matter most in systems engineering are not single-domain problems. There are problems where the coupling between domains determines the answer. A battery thermal management system's control performance depends on the electrochemical model's thermal behavior, which depends on the coolant flow model's heat removal rate, which depends on the pump control strategy, which depends on the battery temperature the controller measures. No single-domain simulation captures this chain.

Modelica is a language specifically designed for this problem. It's acausal, equation-based architecture means a modeler writes physical laws — not signal flow diagrams — and the compiler determines how information flows through the connected system. The same component model works in any system context because the component does not assume which of its variables are inputs and which are outputs. That decision is made at system assembly time, determined by the equations.

5.2 The Modelica Decision Matrix

System type	Modelica strength	Limitation	Supplement with
Multi-domain physical (thermal-mechanical -electrical)	Native strength; acausal coupling handles cross-domain interactions automatically	None significant for lumped-parameter systems	—
Automotive powertrain + BMS + controls	Industry standard; DLR, BMW, Toyota validated libraries	High-frequency switching power electronics	SPICE tools for switching circuits; ROM export via FMI
Building energy and HVAC	Lawrence Berkeley LBNL Buildings library; DOE validated	Full 3D airflow modeling	CFD offline for HTC correlations; feed into Modelica
Aerospace ECS and hydraulics	ThermoSysPro, DLR aerospace libraries; ESA qualified	Spatially distributed structural behavior	ANSYS/Nastran ROM exported as FMU
High-frequency power electronics	Poor fit; solver restarts at every switching event degrade performance	Fundamental	LTspice, Cadence for detailed design; behavioral ROM for system level

5.3 FMI Co-Simulation: Connecting Tools Without Replacing Them

The Functional Mockup Interface (FMI) standard is the integration fabric that allows Modelica thermal models, Simulink control models, ANSYS structural reduced-order models, and AADL software timing models to operate simultaneously in a co-simulation without any tool needing to understand another's internals.

An FMU (Functional Mockup Unit) is a self-contained simulation component — a zip archive containing compiled simulation code and a model description XML that declares every variable the component exposes: name, data type, physical unit, and causality (input, output, parameter, or state). Any conformant host environment reads the XML and knows the component's interface without executing any simulation code.

The master algorithm coordinates the co-simulation, advancing all FMUs through time and exchanging variable values at communication points. Three coordination schemes address different coupling regimes:

- **Jacobi (parallel):** FMUs advance simultaneously using previous-step outputs. Efficient; introduces one-step delay. Appropriate for weakly coupled systems.
- **Gauss-Seidel (sequential):** FMUs update in dependency order, passing fresh outputs immediately downstream. Reduces delay error for asymmetrically coupled systems.
- **Iterative (Newton-Raphson):** Master iterates until inputs and outputs converge at each communication point. Near-monolithic accuracy; k times the computation cost. Required for strongly coupled bidirectional systems.

The communication step size trade-off

The communication step size h between FMU exchanges is the central numerical parameter of co-simulation. The governing rule: h must be at least 10 times smaller than the fastest dynamic time constant at any interface. A thermal-mechanical coupling with a 1-second time constant tolerates $h = 100\text{ms}$. A fast electrical transient with a 1ms time constant requires $h = 0.1\text{ms}$ or domain decoupling. Violating this rule produces trajectories that are numerically smooth but physically wrong — the most dangerous simulation failure mode.

5.4 Thermal Simulation: The Coupling Domain

Thermal simulation occupies a unique position in the MBSE stack because heat is the medium through which most other domains interact. Heat generation depends on electrical dissipation, friction, and chemical reactions. Heat removal depends on fluid flow. The resulting temperature field drives structural thermal expansion, changes material properties and electrical resistance, and determines electrochemical reaction rates. Thermal is simultaneously a receiver from and a provider to every other physics domain.

This coupling determines the integration strategy. One-way thermal-structural coupling — run thermal analysis, export temperature field as a structural body load — is adequate when deformations do not change thermal boundary conditions. Two-way coupling through FMI is required when they do: turbine blade tip clearance changing under thermal expansion modifies convective boundary conditions, which changes the temperature field, which changes the deformation. For system-level co-simulation, thermal lumped-parameter models in Modelica provide the speed needed; FEA thermal analyses provide the spatial detail needed for component-level decisions; reduced-order models bridge between the two.

5.5 From Simulation Output to RSM Input: The Sampling Protocol

MBSE simulation outputs become RSM inputs through a structured sampling protocol. The DoE design specifies the evaluation points — the specific combinations of design variable values at which the MBSE simulation is run. Each run produces the objective function values (maximum temperature, total mass, control bandwidth, reliability metric) that the RSM surface is fitted to.

The practical challenge is that a single full-system co-simulation run may take hours on a multi-core cluster. The DoE design must minimize the number of runs while preserving the information needed to fit a reliable surface. A 20-run CCD for three design variables requires 20 full co-simulation runs — achievable in a week of compute time on a small cluster. A 125-run full factorial is not.

This constraint is also a discipline. The DoE forces the team to specify in advance what design variables are being studied and what objectives are being measured. This specification process surfaces disagreements about what the design space is and what the program is optimizing — conversations that are valuable to have explicitly rather than implicitly.

6. Controls Integration and Hardware Verification: Closing the Loop

Decision outcomes in this section enable:

- Connect control law design to architecture requirements through formally typed interfaces
- Select the right verification stage for each class of control system requirement
- Understand what HiL uniquely reveals and why it cannot be skipped

6.1 The Abstraction Gap and Its Cost

The systems architect's control interface declaration and the controls engineer's Simulink model occupy different worlds. The architecture model says what signals cross the interface, in what units, at what update rates. The Simulink model implements a specific control law connected to a specific plant model with specific numerical parameters. In the absence of a formal connection between these two artifacts, they diverge — and the divergence is invisible until late-phase integration testing, when the cost of fixing it is at its highest.

This is not a tooling failure. It is an information architecture failure. The connection between the architecture model's interface declaration and the simulation model's signal definition is a relationship that must be made explicit, versioned, and checked automatically. SysML v2's formally typed ports — carrying ISO 80000 quantity kinds and physical units — combined with the REST API that makes those declarations queryable, provide the technical foundation for that automated checking.

6.2 The Plant Model Fidelity Problem

The control law is only as good as the plant model it was designed against. A PID controller tuned against a linearized thermal model may behave very differently when connected to the full nonlinear Modelica plant that captures thermal stratification, heat exchanger effectiveness degradation, and pump curve nonlinearity. This is not a problem with the control design method — it is a problem with the verification confidence attributed to it.

The appropriate response is to trace which plant model fidelity level was used to validate each control requirement. This traceability is not documentation overhead — it is the information needed to know which control requirements must be re-verified when the plant model is updated. In a knowledge graph architecture where the simulation configuration, plant model version, requirement, and verification result are all connected nodes, this traceability is a query. Without it, it is a manual audit.

6.3 The Verification Progression: What Each Stage Proves

Stage	Real Hardware?	What It Proves	What It Cannot Prove	Certification Role
MiL	None	Control law behavior against idealized plant under floating-point arithmetic	Code generation artifacts; real-time timing; I/O electrical characteristics	Control law requirements; initial stability margin verification

SiL	Compiled code on PC	Fixed-point arithmetic; quantization effects; code generation correctness	Processor timing; bus loading; I/O hardware behavior	Software unit and integration test evidence
PiL	Real processor only	Interrupt latency; RTOS scheduling; processor-specific cache effects	I/O electrical characteristics; full bus loading under realistic traffic	Timing requirement evidence on target hardware
HiL	Full ECU + RT sim	Closed-loop system with all hardware timing; real I/O; failure mode response	Environmental stress; vibration; full system integration	Subsystem validation evidence; DO-178C / ISO 26262 compliance

6.4 What HiL Uniquely Reveals

Hardware-in-the-loop testing is not simply a higher-fidelity version of SiL. It reveals a specific class of failure modes that no simulation-only stage can detect:

- **Real-time execution violations:** A control loop designed for 10ms execution may average 10.3ms with occasional 12ms spikes on the production processor due to cache effects and interrupt contention. This introduces a phase lag that degrades stability margins. SiL on a PC does not reproduce this.
- **I/O hardware behavior:** Real sensors have output impedance, noise spectral density, and temperature-dependent offset drift that differ from their simulation models. Real actuator drivers have switching transients and current-limiting behaviors that affect closed-loop dynamics.
- **Communication bus loading:** CAN bus arbitration contention under realistic multi-node loading causes message delivery timing variations that a perfect-delivery simulation does not capture.
- **Failure mode injection:** Sensor drift, actuator stall, power supply voltage sag, and bus message loss can be injected controllably and repeatably — enabling systematic verification of fault detection and reconfiguration logic that cannot be safely created in a real vehicle or aircraft.

HiL is the point in the verification progression where the program has its last inexpensive opportunity to find the failure modes that would be expensive, dangerous, or impossible to find in physical system integration. Programs that truncate HiL coverage to save schedule are typically trading a short-term schedule recovery for a late-phase integration risk whose resolution cost is 10 to 100 times larger.

6.5 HiL Evidence in the Decision Architecture

HiL test results are not just verification artifacts — they are decision inputs. A Pareto frontier candidate that has passed HiL verification under all relevant operating conditions and failure modes carries qualitatively higher decision confidence than one that has only been validated against an MiL surrogate. Making this confidence distinction explicit, traceable, and queryable requires the knowledge graph infrastructure described in Part III.

PART III

Operationalizing the Decision Architecture

Program-phase implementation, knowledge graph integration, AI grounding, and competitive outcomes

7. The Decision Architecture Across Program Phases

Decision outcomes in this section enable:

- Know which combination of the three methods applies at each major program decision moment
- Recognize the failure mode signatures of programs not using this architecture
- Structure program reviews around frontier-based decision conversations

7.1 Phase Mapping: What the Triad Does When

Program Phase	Dominant Decision	Primary Method	Supporting Methods	Output
Concept and trade studies	Which system concept is architecturally viable?	Pareto + RSM (lightweight surrogates)	SysML v2 for interface definition	Frontier over concept alternatives; dominated concepts eliminated
Preliminary design	What are the key design parameters and their sensitivities?	RSM (DoE sampling of MBSE simulation)	Pareto for trade-off characterization	Validated surrogate; ranked sensitivity; design space map
Critical design	Does the selected design meet all requirements at all operating conditions?	MBSE simulation (full co-simulation)	SysML v2 requirements traceability	Verification evidence matrix traceable to architecture requirements
Integration and test	Are the real hardware and software performing as the model predicted?	HiL (model vs. hardware correlation)	MBSE model correlation update	Model update with correlated hardware behavior; residual risk characterization

Change impact	What does this change break?	SysML v2 API query + surrogate re-evaluation	Pareto re-sweep if objectives shift	Stale analysis inventory; updated frontier for changed design
---------------	------------------------------	--	-------------------------------------	---

7.2 The Change Impact Case: Where the Architecture Pays the Most

The return on investment from the decision architecture is highest at change impact assessment — the moment when a component is redesigned, a requirement is tightened, or an interface is redefined late in the program. Without the architecture, change impact assessment requires manually identifying every analysis that depends on the changed element, re-running those analyses, and reconciling the results. This process takes weeks and is invariably incomplete — some dependent analysis is always missed.

With the architecture, change impact assessment is a query. The SysML v2 API returns all elements that reference the changed component. The knowledge graph returns all simulations and verification results that were produced against the previous version. The RSM surrogate for the affected subsystem is re-evaluated at the new design point in seconds. If the new point is still on or near the Pareto frontier, the decision is straightforward. If it has moved off the frontier, the design team knows immediately — and can begin the recovery analysis with full information about what specifically broke and by how much.

7.3 Failure Mode Signatures: How to Recognize a Program Without This Architecture

Programs operating without the decision architecture exhibit characteristic pathologies:

- **Analysis synchronization meetings:** Weekly or bi-weekly meetings where discipline leads exchange boundary conditions manually, reconcile conflicting assumptions, and update each other on model changes. Each of these meetings represents decision latency that the architecture would eliminate.
- **Late-phase interface surprises:** Unit mismatches, signal definition conflicts, and timing assumption violations discovered during integration testing rather than at model assembly. These are exactly the errors that formally typed SysML v2 interfaces and automated conformance checking prevent.
- **Design decisions attributed to engineering judgment:** When a program's design record shows key decisions justified by "engineering experience" rather than by a traceable analysis, those decisions were made in the absence of Pareto-grounded trade-off information. They may be correct, but they are not defensible and they cannot be efficiently revisited when conditions change.
- **Verification coverage gaps discovered at audit:** Requirements with no simulation analysis or test case, discovered during a certification audit. These represent the compounded cost of managing verification traceability in documents rather than in a queryable knowledge graph.

8. The Knowledge Graph: Making Traceability Structural

Decision outcomes in this section enable:

- Understand why document-based traceability breaks under the pace of modern program change
- Design the knowledge graph schema that connects all five layers of the decision architecture
- Apply the Circuit Breaker pattern to protect decision quality under AI-assisted workflows

8.1 Why Document-Based Traceability Fails

The decision architecture described in this document generates a large volume of interconnected artifacts: SysML v2 requirements and interface declarations, Modelica and Simulink simulation configurations and results, FMU model descriptions, DoE designs and RSM surrogate parameters, Pareto frontier records with candidate design specifications, HiL test cases and results, and the version relationships connecting all of these to the architecture commits against which they were validated.

Document-based traceability — maintaining these connections in requirement traceability matrices, test reports, and interface control documents — breaks under the pace of modern program change for a simple reason: documents cannot be automatically updated when the artifacts they reference change. When a requirement changes in the SysML model, no document knows it has become stale. When a simulation is re-run with an updated plant model, the document recording the previous result does not update. The traceability fabric degrades in proportion to program pace.

8.2 The Knowledge Graph Schema

The knowledge graph replaces the document fabric with a typed, directed graph where nodes are artifacts and edges are relationships between them. The schema for the decision architecture has five primary node types:

Node type	Source	Key attributes	Connected to
ArchitectureElement	SysML v2 REST API	UUID, type, name, commit version, constraint expressions	RequirementSatisfaction, SimulationInterface, VerificationResult
SimulationConfiguration	FMU model descriptions + DoE records	Tool versions, parameter values, FMU versions, operating conditions	ArchitectureElement (interfaces), SurrogateModel, VerificationResult
SurrogateModel	RSM fitting records	Polynomial coefficients, DoE design, validation metrics, valid bounds	SimulationConfiguration (training data), ParetoFrontier
ParetoFrontier	NSGA-II / optimizer records	Objective values, design variable values, commit timestamp	SurrogateModel, ArchitectureElement (requirements), VerificationResult

VerificationResult	HiL test records + simulation results	Pass/fail, timestamp, operator, configuration version	ArchitectureElement (requirement), SimulationConfiguration, ParetoFrontier
--------------------	---------------------------------------	---	--

With this schema, the decisions a program needs to make are queries. "Which Pareto frontier candidates have been HiL-verified against the current architecture commit?" is a three-hop graph traversal. "Which requirements have no passing verification result against any simulation configuration built from the current SysML v2 model version?" is a two-hop traversal with a NOT EXISTS filter. These queries take milliseconds. Their document-based equivalents take days.

8.3 The Circuit Breaker Pattern for Decision Quality

As AI assistants are increasingly deployed to support engineering workflows — recommending simulation configurations, interpreting Pareto frontiers, generating SysML model fragments, explaining HiL test anomalies — a new failure mode emerges: AI-generated assertions that are plausible and confidently stated but inconsistent with the verified engineering record.

The Circuit Breaker pattern addresses this by grounding AI outputs against the knowledge graph before they reach engineers. The mechanism: before an AI assistant makes a claim about system architecture, interface specifications, simulation behavior, or verification status, it retrieves the relevant nodes from the knowledge graph and checks its assertion against the retrieved ground truth. Discrepancies trigger a hold rather than silent propagation.

This is not a general AI safety concern — it is a specific engineering quality concern. In a program context, an AI that confidently recommends a simulation configuration that is inconsistent with the current SysML v2 interface declaration is creating exactly the kind of interface mismatch that costs weeks to find and fix. The Circuit Breaker prevents that cost by inserting a verification step before the misconfiguration propagates.

Mean Time Between Hallucinations (MTBH)

MTBH is a proposed metric for AI grounding effectiveness in engineering decision workflows: the rate at which AI-generated engineering assertions that were not validated against the knowledge graph later prove inconsistent with verified system specifications. The metric is measurable because the knowledge graph provides an auditable ground truth. Lower MTBH correlates with better grounding discipline and higher confidence in AI-assisted decision speed gains. Programs that instrument this metric can quantify both the speed benefit of AI assistance and the quality cost of insufficient grounding.

9. Competitive Advantage: What This Architecture Makes Possible

Decision outcomes in this section enable:

- Articulate the program-level outcomes that the decision architecture enables
- Identify the bid and proposal differentiators that this capability creates
- Understand the maturity progression for programs at different starting points

9.1 What Becomes Possible

The decision architecture is not valuable because it uses sophisticated methods. It is valuable because of what it enables that would otherwise be impossible or prohibitively expensive:

Capability	Without the architecture	With the architecture
Design space exploration	Evaluate 5-10 design alternatives per phase; choose by judgment	Evaluate 1,000+ alternatives via surrogate; choose from Pareto frontier
Change impact assessment	2-4 weeks of analysis re-running and reconciliation	Hours: graph query identifies stale analyses; surrogate re-evaluates immediately
Requirements verification	Manual comparison of simulation results against requirement documents	Automated: SysML v2 constraint expressions checked against simulation outputs
Multi-discipline trade-offs	Negotiated in review meetings; outcome depends on organizational dynamics	Characterized on a Pareto frontier; stakeholders choose with explicit trade-off awareness
Model-to-hardware correlation	Discovered at integration test; retroactive model corrections	HiL correlated against MBSE model; model updated proactively before integration
AI-assisted engineering	AI suggestions lack grounding; quality uncertain; verification burden on engineers	AI claims validated against knowledge graph before presentation; grounded acceleration

9.2 The Bid and Proposal Differentiator

Programs operating with the decision architecture can make claims in bids and proposals that programs without it cannot credibly make. These claims address the customer's most common risk concerns directly:

- **Schedule confidence:** "Our design space was characterized by a validated surrogate model across 847 design configurations before PDR. Our schedule reflects that we have already eliminated the dominated designs and understand the trade-space of the remaining candidates."

- **Late-change resilience:** "Our architecture model is the versioned, queryable source of truth for all downstream simulations and verification evidence. When requirements change, we can identify all affected analyses within hours and estimate re-verification cost before committing to a schedule impact."
- **Verification completeness:** "Every Pareto frontier candidate presented at CDR carries a requirements traceability chain from the design variable values through the simulation configuration and RSM surrogate to the formal SysML v2 requirement it satisfies."
- **Integration risk:** "Our HiL program verified the control system against a co-simulation plant model built from validated physics FMUs. The model-to-hardware correlation data is in the program knowledge graph. We know where the model is accurate and where residual uncertainty exists before aircraft integration."

9.3 Implementation Maturity Progression

Programs at different starting points enter the maturity progression at different stages. The progression is cumulative — each level builds on the previous — and each level delivers measurable decision quality improvement even without the levels above it:

Maturity level	Capability	Immediate decision benefit	Enabling technology
Level 1: Formal interfaces	SysML v2 interface declarations with typed quantity kinds and units	Automated interface conformance checking; unit mismatch prevention	SysML v2 tool + REST API access
Level 2: Surrogate pipeline	DoE-sampled MBSE simulation + RSM surrogate per discipline	Design space exploration at 50-100x lower cost; sensitivity ranking	Modelica/Simulink + DoE software + Python/R RSM fitting
Level 3: Pareto integration	Multi-objective optimizer sweeping validated surrogates	Pareto-grounded trade-off conversations at design reviews; dominated designs eliminated	NSGA-II implementation (pymoo, MATLAB) + multi-surrogate interface
Level 4: Knowledge graph	Traceability graph connecting all artifacts through typed relationships	Change impact in hours; verification gap detection before audit; version-aware stale analysis	Neo4j or RDF store + SysML v2 API ingestion + FMU manifest parsing
Level 5: AI grounding	Circuit Breaker pattern; MTBH instrumentation	AI-assisted decision speed with verifiable quality; continuous grounding discipline improvement	LLM integration with knowledge graph query; MTBH monitoring

The most important transition is from Level 1 to Level 2 — from formal interfaces to a working surrogate pipeline. Programs that have formally typed interface declarations but no surrogate model have done the definitional work without the acceleration work. The surrogate pipeline is where the decision tempo actually changes. Everything above Level 2 amplifies that acceleration and extends it across program phases.

10. Conclusion: The Decision Architecture as Engineering Practice

The three methods examined in this document — MBSE simulation, Response Surface Methods, and Pareto Optimality Analysis — are individually well-established and individually valuable. The argument of this document is that their value multiplies when they are used as a unified decision architecture, connected through formal interfaces, versioned repositories, and a traceability knowledge graph.

For chief architects and systems engineers, the practical commitment this architecture requires is not primarily a tooling commitment. The tools exist. SysML v2 tool implementations are maturing. Modelica simulators are industrial-grade. FMI is a stable standard. RSM and Pareto optimization are available in open-source packages. The commitment is an information architecture commitment: to treat the connections between methods — interface declarations, simulation configurations, surrogate parameters, verification traceability — as first-class engineering artifacts that require the same discipline, versioning, and maintenance as the models themselves.

Programs that make that commitment gain something that cannot be bought with additional headcount or compute time: the ability to answer consequential engineering questions in hours rather than weeks, with confidence that the answer is grounded in validated physics and traceable to formal requirements. That capability is the source of the schedule confidence, the change resilience, the verification completeness, and the competitive differentiation described in Section 9.

The programs that will define engineering best practice in the next decade are not the ones that run the most sophisticated individual simulations. They are the ones who have built the architecture to make those simulations speak to each other, speak to their requirements, and speak to the decision-makers who need to act on what they say.

The binding constraint on engineering program performance is not capability.

It is decision latency.

The decision architecture collapses that latency without sacrificing rigor.

Appendix: Quick Reference

A. Method Selection Matrix

If your decision is about...	Lead with	Support with
Which system concept to pursue	Pareto (lightweight surrogates over concepts)	SysML v2 for requirements; RSM for objective functions
Sensitivity of performance to design parameters	RSM (DoE sampling + surface fitting)	MBSE simulation as the evaluation oracle
Optimal values for continuous design parameters	RSM + gradient-based optimizer	Pareto if multiple objectives; MBSE simulation for verification
Trade-offs between conflicting system objectives	Pareto (NSGA-II frontier)	RSM surrogates for efficiency; MBSE for frontier verification
Whether the design meets requirements at all conditions	MBSE co-simulation (full physics)	SysML v2 requirements traceability; HiL for hardware fidelity
Impact of a proposed design change	SysML v2 API query + RSM re-evaluation	Pareto re-sweep if objectives shift
Whether the real hardware matches the model	HiL correlation test	MBSE model update with correlated behavior

B. Key Acronyms

Acronym	Expansion	Section reference
AADL	Architecture Analysis and Design Language (SAE AS5506)	Section 4.3
CCD	Central Composite Design	Section 2.3
DoE	Design of Experiments	Section 2.3
EMO	Evolutionary Multi-objective Optimization	Section 3.2
FMI	Functional Mockup Interface	Section 5.3
FMU	Functional Mockup Unit	Section 5.3
HiL	Hardware-in-the-Loop	Section 6.4
HTC	Heat Transfer Coefficient	Section 5.4
KerML	Kernel Modeling Language (SysML v2 foundation)	Section 4.2

MBSE	Model-Based Systems Engineering	Throughout
MiL	Model-in-the-Loop	Section 6.3
MTBH	Mean Time Between Hallucinations	Section 8.3
NSGA-II	Non-dominated Sorting Genetic Algorithm II	Section 3.2
Pareto	Pareto Optimality Analysis	Section 3
PiL	Processor-in-the-Loop	Section 6.3
POD	Proper Orthogonal Decomposition (ROM method)	Section 5.4
ROM	Reduced-Order Model	Section 5.4
RSM	Response Surface Methods	Section 2
SiL	Software-in-the-Loop	Section 6.3
SysML	Systems Modeling Language (OMG standard)	Section 4
WCET	Worst-Case Execution Time (AADL property)	Section 4.3